

Nama : Midori Harahap

NIM : J0403251018

Kelas : TPL A2

Praktikum 1 - Membuat Adjacency Matrix

Source Code Python	<pre>praktikum1.py > ... 1 #----- 2 # Praktikum 1 3 # Nama = Midori Harahap 4 # NIM = J0403251018 5 # Kelas = TPL A2 6 #----- 7 8 def createGraph(V, edges): 9 """ 10 Fungsi untuk membuat adjacency matrix 11 """ 12 #Membuat matrix berukuran V x V dan semua isi awalnya 0 (belum ada hubungan antar node) 13 mat = [[0 for _ in range(V)] for _ in range(V)] 14 15 #Perulangan untuk setiap edge pada list edges 16 for it in edges: 17 u = it[0] #Mengambil node awal 18 v = it[1] #Mengambil node tujuan 19 mat[u][v] = 1 #Menandai bahwa node u terhubung ke node v 20 mat[v][u] = 1 #Karena graph bersifat undirected (dua arah) maka v juga terhubung ke u 21 return mat #Mengembalikan adjacency matrix yang sudah dibuat 22 23 #Program utama 24 if __name__ == "__main__": 25 V = 4 #Jumlah vertex dalam graph 26 27 #Daftar edge antar vertex 28 edges = [[0,1], [0,2], [1,2], [3,2]] 29 30 #Membuat graph dalam bentuk adjacency matrix 31 mat = createGraph(V, edges) 32 33 #Menampilkan output 34 print("Adjacency Matrix Representation:") 35 for i in range(V): #Perulangan untuk menampilkan matrix 36 for j in range(V): #Perulangan kolom pada setiap baris 37 print(mat[i][j], end=" ") #Menampilkan nilai matrix 38 print() #Pindah baris setelah semua kolom ditampilkan</pre>
Hasil program	<pre>PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS DEVDB PS C:\KULIAH\Algoritma dan Struktur Data\Pertemuan11_Midori Harahap_J0403251018> py praktikum1.py Adjacency Matrix Representation: 0 1 1 0 1 0 1 0 1 1 0 1 0 0 1 0</pre>

Praktikum 2 - Membuat Adjacency List

Source
Code
Python

praktikum2.py > createGraph

```
1 #-----
2 # Praktikum 1
3 # Nama = Midori Harahap
4 # NIM = J0403251018
5 # Kelas = TPL A2
6 #-----
7
8 def createGraph(edges):
9     """
10     Fungsi untuk membuat adjacency list
11     """
12     adj = {} #Dictionary kosong untuk menyimpan graph
13
14     #Melakukan perulangan pada setiap edge
15     for it in edges:
16         u = it[0] #Mengambil node asal
17         v = it[1] #Mengambil node tujuan
18
19         if u not in adj: #Jika node u belum ada di dictionary maka buat list kosong dulu
20             adj[u] = []
21
22         if v not in adj: #Jika node v belum ada di dictionary maka buat list kosong dulu
23             adj[v] = []
24
25         adj[u].append(v) #Menambahkan v ke daftar tetangga u
26         adj[v].append(u) #Menambahkan u ke daftar tetangga v
27     return adj #Mengembalikan adjacency list
28
29 #Program utama
30 if __name__ == "__main__":
31     V = 4
32
33     #List edge antar node menggunakan huruf sebagai node
34     edges = [
35         ["A", "B"],
36         ["A", "C"],
37         ["B", "D"],
38         ["C", "D"]
39     ]
40
41     adj = createGraph(edges) #Membuat graph
42
43     print("Adjacency List Representation:") #Menampilkan adjacency list
44     #Perulangan untuk setiap node pada dictionary
45     for i in adj:
46         print(f"{i}: ", end=" ") #Menampilkan nama node
47         for j in adj[i]: #Menampilkan semua tetangga node
48             print(j, end=" ")
49         print()
```

Hasil
Program

```
PS C:\KULIAH\Algoritma dan Struktur Data\Pertemuan11_Midori Harahap_J0403251018> py praktikum2.py
Adjacency List Representation:
A: B C
B: A D
C: A D
D: B C
```

Praktikum 3 - Konversi Matrix ke List

Source
Code
Python

```
praktikum3.py > matrixToList

1  #-----
2  # Praktikum 3
3  # Nama   = Midori Harahap
4  # NIM    = J0403251018
5  # Kelas  = TPL A2
6  #-----
7
8  #Konversi Matrix ke List
9  Windsurf: Refactor | Explain | X
10 def matrixToList(matrix):
11     """
12     Fungsi mengubah adjacency matrix ke adjacency list
13     """
14     adj = {} #Membuat dictionary kosong untuk menyimpan adjacency list
15     V = len(matrix) #Mengambil jumlah vertex
16
17     #Perulangan untuk setiap baris matrix
18     for i in range(V):
19         adj[i] = [] #Membuat list kosong untuk tetangga node i
20
21     #Perulangan untuk setiap kolom
22     for j in range(V):
23         if matrix[i][j] == 1: #Jika bernilai 1 berarti ada edges
24             adj[i].append(j) #Tambahkan node j ke adjacency list node i
25     return adj #Mengembalikan adjacency list
26
27 #Program utama
28 if __name__ == "__main__":
29
30     #Adjacency matrix yang mau diubah
31     matrix = [
32         [0,1,1,0],
33         [1,0,1,0],
34         [1,1,0,1],
35         [0,0,1,0]
36     ]
37
38     adj = matrixToList(matrix) #Mengubah matrix menjadi list
39
40     print("Adjacency List Representation:") #Menampilkan hasil adjacency list
41     for i in adj: #Perulangan setiap node
42         print(f"{i}: ", end=" ") #Menampilkan node
43
44         for j in adj[i]: #Menampilkan tetangga node
45             print(j, end=" ")
46         print()
47
48
```

Hasil
Program

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  DEVD
PS C:\KULIAH\Algoritma dan Struktur Data\Pertemuan11_Midori Harahap_J0403251018> py praktikum3.py
Adjacency List Representation:
0:  1 2
1:  0 2
2:  0 1 3
3:  2
```

Praktikum 4 - Studi Kasus Dunia Nyata

Source
Code
Python

```
praktikum4.py >...
1 #-----
2 # Praktikum 4
3 # Nama    = Ridori Harahap
4 # NIM     = 38403251818
5 # kelas   = TP1-A2
6 #-----
7
8 #Studi Kasus Dunia Nyata
9
10 #-----
11 #Langkah 1 - Tentukan Studi Kasus
12 #-----
13 #NIM 38403251818 maka studi kasusnya Peta Kota
14
15 #-----
16 #Langkah 2- Tentukan Node dan Edge
17 #-----
18 #Vertex
19 #Bogor, Jakarta, Depok, Tangerang, Bekasi
20
21 #Edge
22 #Bogor <-> Jakarta
23 #Bogor <-> Depok
24 #Jakarta <-> Tangerang
25 #Jakarta <-> Bekasi
26 #Depok <-> Bekasi
27
28 #-----
29 #Langkah 3 - Gambar Desain Graph
30 #-----
31
32 #-----
33 #Langkah 4 - Implementasi dalam Python
34 #-----
35
36 #Daftar kota (vertex)
37 perkotaan = ["Bogor", "Jakarta", "Depok", "Tangerang", "Bekasi"]
38
```

```
39
40 V = len(perkotaan) #Jumlah vertex
41
42 #Dictionary untuk idex kota agar nama kota bisa diakses menggunakan angka
43 indeks_kota = {
44     "Bogor": 0,
45     "Jakarta": 1,
46     "Depok": 2,
47     "Tangerang": 3,
48     "Bekasi": 4
49 }
50
51 #Daftar jalan antar kota (edge)
52 jalan = [
53     ("Bogor", "Jakarta"),
54     ("Bogor", "Depok"),
55     ("Jakarta", "Tangerang"),
56     ("Jakarta", "Bekasi"),
57     ("Depok", "Bekasi")
58 ]
59
60 #-----
61 #Membuat Adjacency Matrix
62 #-----
63 matrix = [[0 for _ in range(V)] for _ in range(V)] #Membuat matrix kosong ukuran V x V
64
65 for u, v in jalan:
66     #Menambahkan edge ke matrix
67     #Mengubah nama kota menjadi index angka
68     i = indeks_kota[u]
69     j = indeks_kota[v]
70     #Menambahkan dua arah (undirected)
71     matrix[i][j] = 1
72     matrix[j][i] = 1
73
```

```
praktikum4.py >...
74 #-----
75 #Membuat Adjacency list
76 #-----
77 adj = {} #Membuat dictionary kosong
78
79 #Membuat list kosong untuk setiap kota
80 for kota in perkotaan:
81     adj[kota] = []
82
83 #Menambahkan hubungan antarkota
84 for u, v in jalan:
85     adj[u].append(v) #Menambahkan kota tujuan ke kota asal
86     adj[v].append(u) #Menambahkan kota asal ke kota tujuan karena undirected
87
88 #-----
89 #Langkah 5 - Tampilkan Output Program
90 #-----
91
92 #Menampilkan nama node
93 #-----
94 print("DAFTAR NAMA KOTA")
95 for kota in perkotaan:
96     print("-", kota)
97 print()
98
99 #Menampilkan hubungan antar node
100 #-----
101 print("HUBUNGAN ANTAR KOTA")
102 for u, v in jalan:
103     print(f"{u} <-> {v}")
104 print()
105
```

```
praktikum4.py >...
106 #Menampilkan adjacency matrix
107 #-----
108 print("ADJACENCY MATRIX")
109 print(" ", end=" ")
110 for kota in perkotaan:
111     print(f"{kota:12}", end=" ")
112 print()
113
114 #Menampilkan list matrix
115 for i in range(V):
116     print(f"{perkotaan[i]:12}", end=" ") #Menampilkan nama kota pada baris
117
118     #Menampilkan isi matrix
119     for j in range(V):
120         print(f"{matrix[i][j]:12}", end=" ")
121     print()
122
123 #Menampilkan adjacency list
124 #-----
125 print("ADJACENCY LIST")
126 #Menampilkan adjacency list tiap kota
127 for kota in adj:
128     print(f"{kota}: ", end=" ")
129
130     for j in adj[kota]:
131         print(j, end=" ")
132     print()
133
```

Hasil Program	<pre> PROBLEMS OUTPUT DEBUG CONSOLE <u>TERMINAL</u> PORTS DEVDB PS C:\KULIAH\Algoritma dan Struktur Data\Pertemuan11_Midori Harahap_J0403251018> py praktikum4.py DAFTAR NAMA KOTA - Bogor - Jakarta - Depok - Tangerang - Bekasi HUBUNGAN ANTAR KOTA Bogor <--> Jakarta Bogor <--> Depok Jakarta <--> Tangerang Jakarta <--> Bekasi Depok <--> Bekasi ADJACENCY MATRIX Bogor Jakarta Depok Tangerang Bekasi Bogor 0 1 1 0 0 Jakarta 1 0 0 1 1 Depok 1 0 0 0 1 Tangerang 0 1 0 0 0 Bekasi 0 1 1 0 0 ADJACENCY LIST Bogor: Jakarta Depok Jakarta: Bogor Tangerang Bekasi Depok: Bogor Bekasi Tangerang: Jakarta Bekasi: Jakarta Depok PS C:\KULIAH\Algoritma dan Struktur Data\Pertemuan11_Midori Harahap_J0403251018> </pre>
Gambar	<pre> graph TD Bogor --- Jakarta Bogor --- Depok Jakarta --- Bogor Jakarta --- Tangerang Jakarta --- Bekasi Depok --- Bogor Depok --- Bekasi Tangerang --- Jakarta Bekasi --- Jakarta Bekasi --- Depok </pre>
Hasil Analisis	<pre> # Graph pada kasus peta jalan dengan kota sebagai verteks dan jalan sebagai edge # Representasi dengan adjacency matrix digunakan untuk melihat hubungan antar kota dengan mengecek nilai 0 atau 1 pada matriks # Representasi dengan adjacency list hanya menampilkan daftar kota yang benar-benar terhubung </pre>
Kesimpulan	<pre> Maka, pada studi kasus ini yang paling efisien bagi user adalah adjacency list karena menampilkan jumlah jalan lebih sedikit sehingga mudah dipahami user </pre>